Inteligență Artificială:

Tema 1 - Introducere în Machine Learning

Documentație

Student: Oncioiu Miruna Elena

Grupa: 332CC

1. Introducere

Prin intermediul acestei teme, am pus in practica notiunile de invatare automata prezentate la curs, construind o solutie completa pentru o problema de inteligenta artificiala. Obiectivul a fost sa analizez un set de date brut, sa identific problemele acestuia (date lipsa, dezechilibre, atribute redundante) si sa il pregatesc pentru antrenare. Ulterior, am implementat, testat si comparat mai multi algoritmi de clasificare si regresie, cu scopul de a prezice cat mai precis tipul de vacanta si, respectiv, salariul angajatilor din setul de date. Evaluarea performantei s-a facut prin jurnalizarea atenta a experimentelor si a metricilor specifice fiecarui algoritm.

3.1 Explorarea Datelor (EDA)

1. Analiza tipului de atribute si a plajei de valori

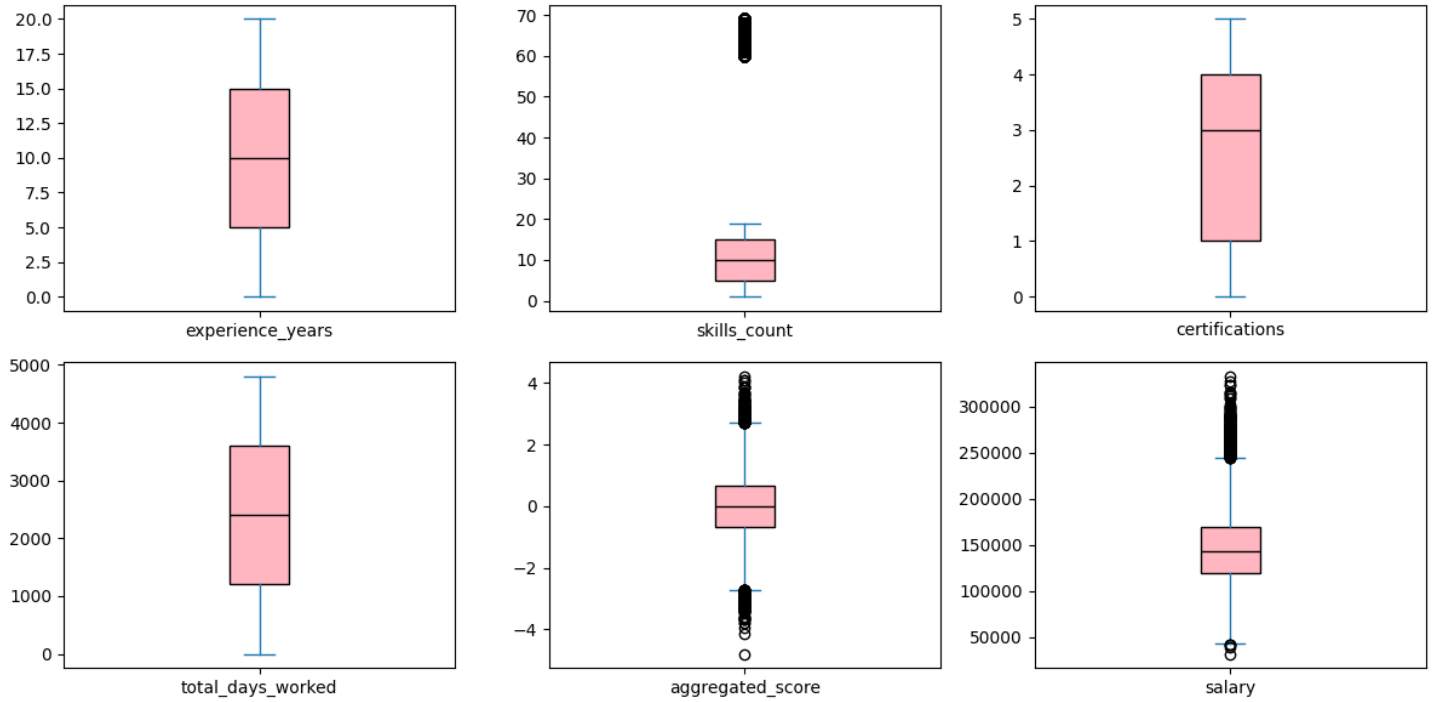
A. Atribute Numerice Continue

Am extras statisticile de baza pentru a observa distributia datelor si prezenta anomaliilor (nr de exemple care nu au valori lipsa, valoarea medie, deviatia standard a valorilor, valoarea minima, valoarea percentilei de 25%, valoarea percentilei de 50% (valoarea mediana), valoarea percentilei de 75%, valoarea maxima. Am introdus si nr de exemple care au valori lipsa, deoarece la urmatorul exerciu (3.2.1) va fi nevoie sa le imputez si am dorit sa le aflu de acum.

	experience_years	skills_count	certifications	total_days_worked	aggregated_score	salary
count	64000.000	64000.000	64000.000	64000.000	64000.000	64000.000
mean	9.973	11.604	2.511	2393.508	0.001	145683.566
std	6.051	10.737	1.710	1452.045	1.000	37384.756
min	0.000	1.000	0.000	-1.000	-4.797	31867.000
25%	5.000	5.000	1.000	1200.000	-0.674	119209.750
50%	10.000	10.000	3.000	2400.000	0.001	143299.000
75%	15.000	15.000	4.000	3600.000	0.678	169582.250
max	20.000	69.000	5.000	4800.000	4.218	333046.000
valori_lipsa	0.000	0.000	0.000	0.000	0.000	0.000

Analizand cifrele extrase in tabelul de statistici, putem remarca o simetrie foarte buna pentru atribute precum experience_years, certifications si total_days_worked. Pentru acestea, media este aproape identica cu valoarea mediana (percentila 50%). De exemplu, experienta medie a angajatilor este de 9.97 ani. Pe de alta parte, in cazul coloanei skills_count, desi 75% dintre angajati detin maxim 15 skill-uri (conform percentilei

75%), exista un maxim absolut de 69. Acest aspect explica aglomerarea de outliers din grafic si indica prezenta unui grup restrans de angajati supraspecializati. In cazul variabilei tinta pentru regresie (salary), media generala se situeaza la valoarea de 145.683, insa asimetria este trasa in sus de acele salarii de exceptie care ajung pana la pragul maxim de 333.046. Acelasi fenomen de valori extreme izolate se observa si in graficul pentru aggregated_score, atat in zona pozitiva cat si in cea negativa. Toate aceste observatii intaresc decizia tehnica de a trata valorile extreme prin metoda IQR in faza de preprocesare, asigurand astfel modele predictive cat mai stabile.



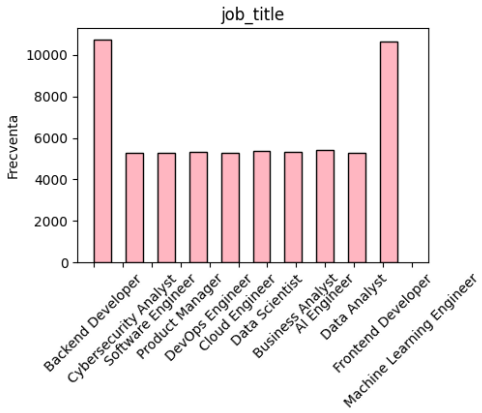
In urma analizei statisticilor descriptive si a diagramelor Boxplot, am observat ca majoritatea atributelor (experience_years, skills_count, today_days_worked, aggregated_score si salary) au o distributie relativ uniforma. Totusi, atributul salary prezinta numeroase valori extreme, vizibile clar deasupra diagramelor Boxplot. Valori extreme mai au si aggregated_score, dar si skills_count. Deoarece algoritmii de regresie sunt sensibili la aceste valori, voi aplica o metoda de filtrare (IQR) in etapa de preprocesare. De asemenea, se observa ca nu exista valori lipsa (NaN) in coloanele numerice, ce trebuie imputate.

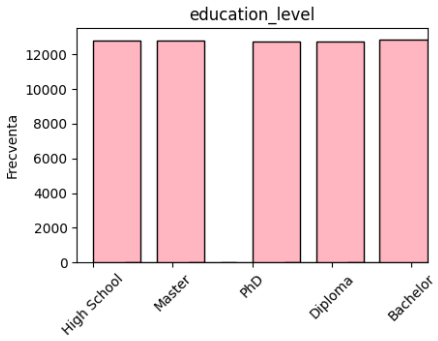
B. Atribute Categorice sau Ordinale

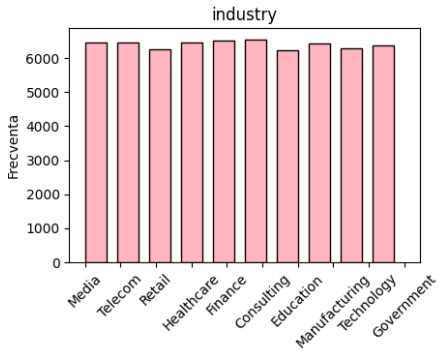
Am analizat numarul de exemple din setul de date care nu au valori lipsa, numarul de valori unice, dar si numarul de valori lipsa.

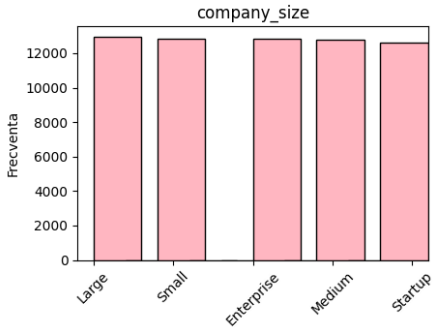
	count-fara lipsa	valori unice	valori lipsa
job_title	64000	12	0
education_level	64000	5	0
industry	64000	10	0
company_size	64000	5	0
location	64000	10	0
remote_work	44847	3	19153
skill_bracket	64000	3	0

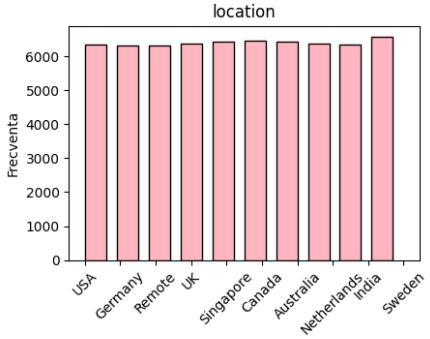
Tabelul de statistici indica un set de date categorice foarte curat, singura exceptie majora fiind atributul remote_work. Acesta prezinta 19 153 de valori lipsa raportat la un total de 64 000 de instante , ceea ce reprezinta o proportie semnificativa si va necesita o strategie de imputare in fata de preprocesare. Celelalte attribute sunt complet populate.

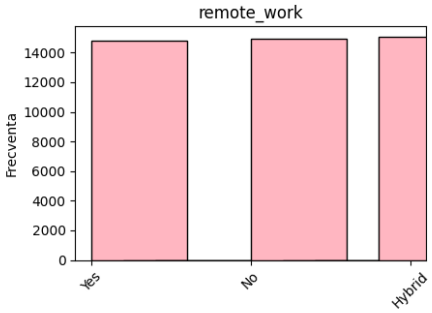


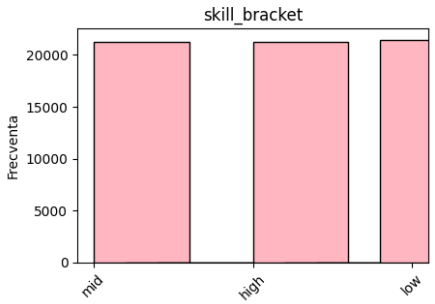












Analizand histogramele, se observa o uniformitate ridicata a distributiei pentru majoritatea categoriilor, precum education_level, company_size, location si skill_bracket, ceea ce indica un set de date foarte bine

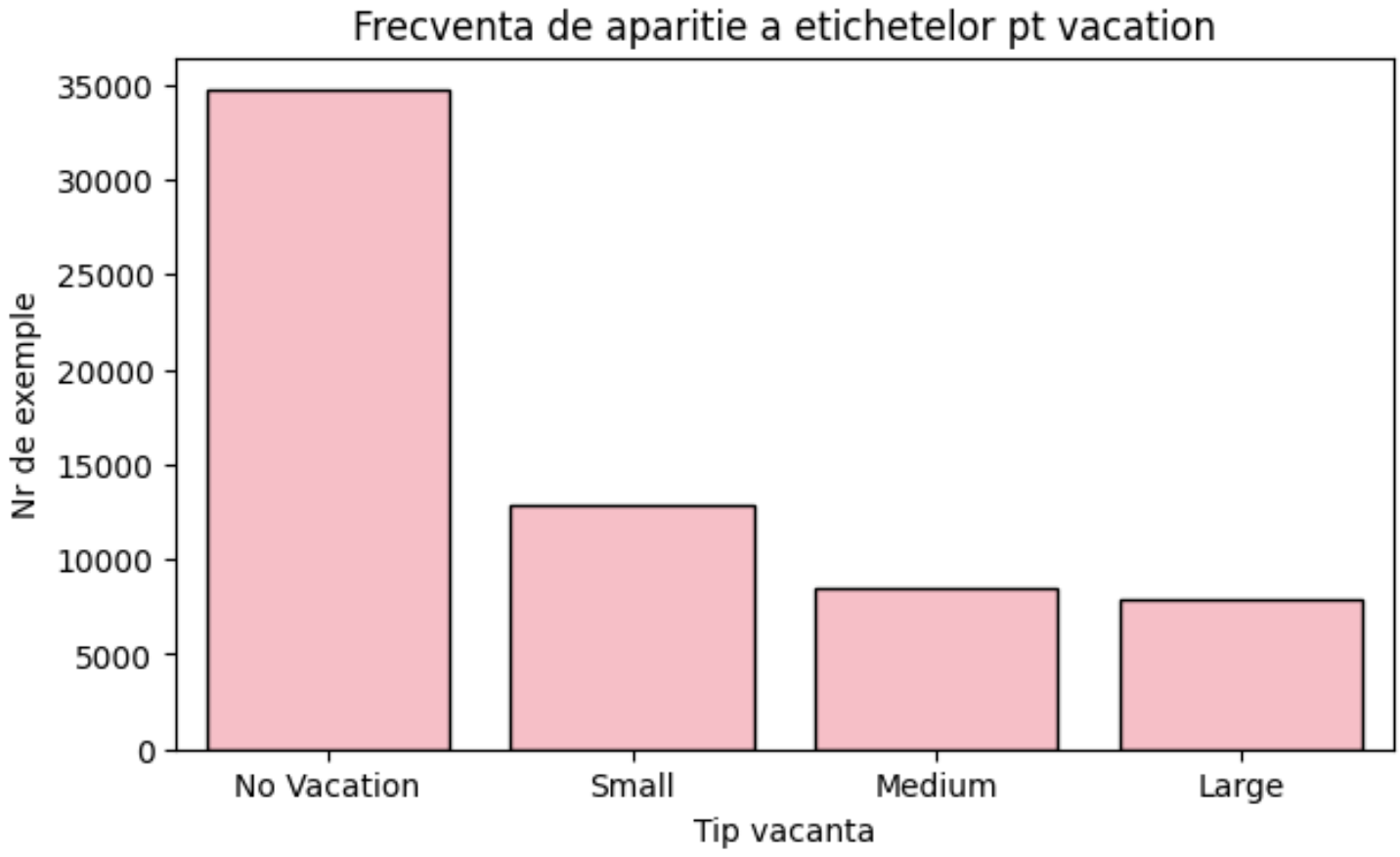
echilibrat. O exceptie apare la atributul job_title, unde rolurile de Backend Developer si Machine Learning Engineer domina clar, avand o frecventa dubla fata de celelalte profesii.

2. Analiza echilibrului de clase si a distributiilor

Pentru a intelege mai bine cum arata variabilele pe care trebuie sa le prezic, am generat doua grafice folosind biblioteca seaborn si matplotlib (un countplot pentru clasificare si un histplot cu functia pentru regresie).

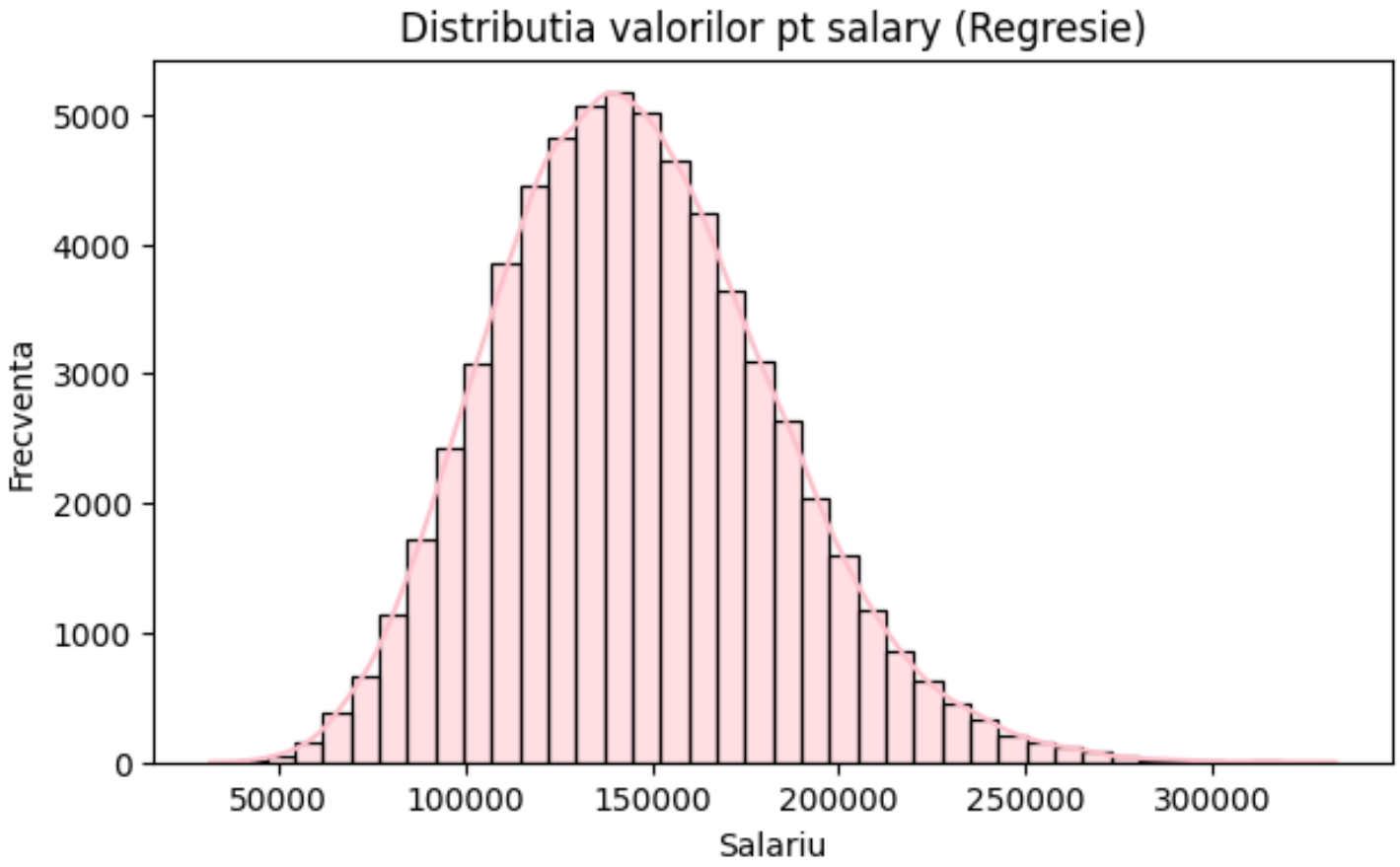
Pe langa analiza vizuala, pentru a avea o imagine completa si clara asupra distributiei, am extras numarul exact de instante pentru fiecare categorie a variabilei tinta de clasificare, vacation. Aceasta verificare confirma vizualizarea din grafic si subliniaza gradul de dezechilibru al setului de date, oferind cifrele concrete pe care ma voi baza in deciziile ulterioare de modelare:

```
vacation
No Vacation    34686
Small          12877
Medium          8527
Large           7910
Name: count, dtype: int64
```



Pentru Clasificare (Tip vacanta): Am studiat frecventa etichetelor pentru variabila tinta de clasificare, vacation. Uitandu-ma la diagrama, este evident ca este un set de date cu un dezechilibru major de clase. Clasa "No Vacation" domina setul de date, avand mai mult de jumatate din exemple, in timp ce clasa "Large" este foarte slab reprezentata.

Conform teoriei, intr-un set de date atat de dezechilibrat, nu ne putem baza deloc pe acuratete (Accuracy) pentru a evalua un algoritm. Un model complet neantrenat, care prezice mereu doar clasa majoritara "No Vacation", ar obtine din start o acuratete artificiala de peste 50%, desi el nu a invatat de fapt nimic despre celelalte clase. De aceea, in sectiunea de Evaluare (3.3), am decis sa ignor acuratetea generala si sa judec performanta modelelor uitandu-ma strict la metricile care rezista la dezechilibru**:** Precizie, Recall si scorul F1. Aceasta abordare imi va permite sa evaluez corect capacitatea modelului de a identifica clasele cu suport scazut.



Pentru a analiza variabila tinta a problemei de regresie, am generat o histograma (cu o curba de densitate KDE suprapusa), reprezentand distributia valorilor salariale din setul de date.

Uitandu-ma la axa orizontala (Salariu), se observa clar ca datele sunt in forma lor bruta, neschimbate matematic (nemodificate prin standardizare), cu valori care variaza aproximativ de la 50.000 pana la 300.000.

Forma distributiei este de clopot, foarte asemanatoare cu o distributie normala (gaussiana), fiind doar usor asimetrica spre dreapta. O mare parte din angajati (cea mai inalta zona a clopotului) se concentreaza in jurul salariului mediu de ~140.000 - 150.000. Totusi, se observa o usoara asimetrie spre dreapta, graficul intinzandu-se subtire spre valori de peste 300.000. Acest lucru confirma vizual existenta acelor valori extreme pe care le-am identificat anterior in diagrama Boxplot. Din punct de vedere al Machine Learning-ului, faptul ca nucleul datelor urmeaza o distributie normala este un avantaj major, in special pentru Regresia Liniara, deoarece favorizeaza un proces de invatare stabil. In acelasi timp, asimetria din dreapta ne valideaza decizia de a trata valorile extreme in etapa de preprocesare.

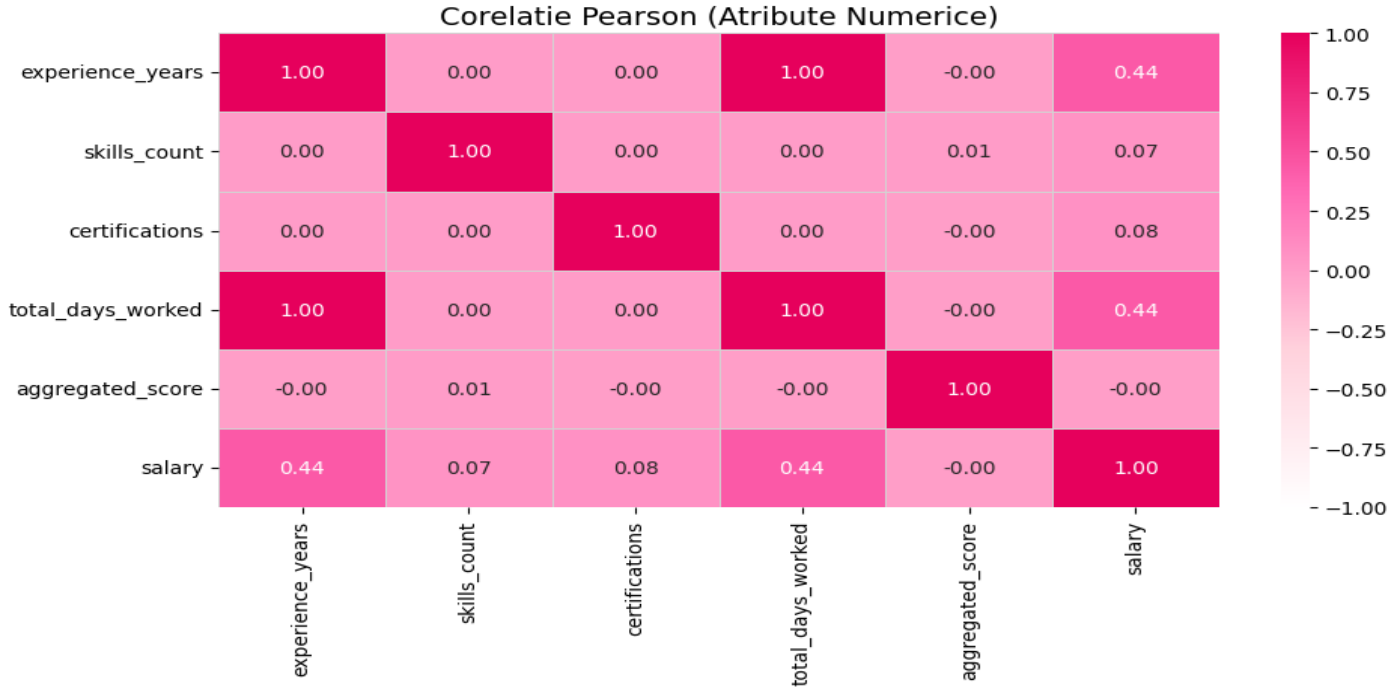
3. Analiza corelatiei intre atribute

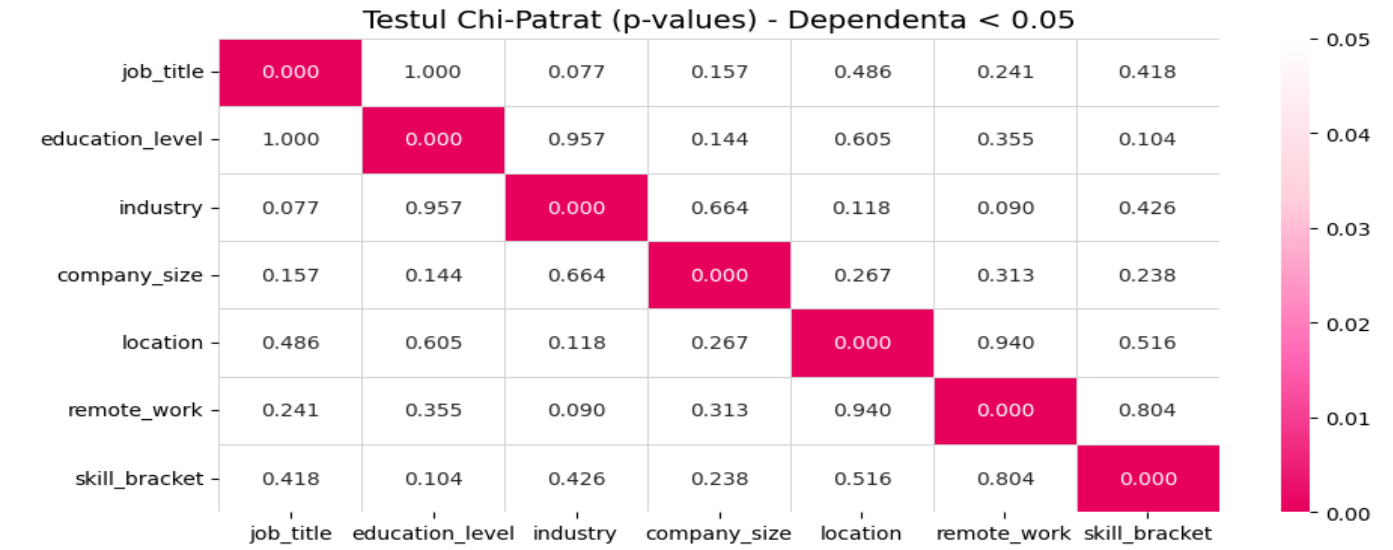
Ultimul pas din explorarea datelor (EDA) a fost sa analizez legaturile dintre variabile. Aici am urmarit doua mari obiective:

1. Sa gasesc atribute redundante (care ofera aceeasi informatie) pentru a le elimina si a scadea complexitatea algoritmilor.
2. Sa testez relatia directa dintre anumiti predictori si variabilele tinta (salariul si vacanta) conform cerintei minime.

3. Identificarea atributelor redundante

Deoarece sunt atat date numerice, cat si categorice, a trebuit sa aplic doua metode statistice distincte pentru a evalua corelatiile. Acest pas a fost crucial pentru a identifica eventualele redundante si pentru a simplifica viitoarele modele predictive, dar si pentru a intelege relatiile directe cu variabilele tinta.





Corelatia numerica (Matricea Pearson): Pentru atributele numerice continue, am calculat coeficientul de corelatie Pearson. Analizand primul heatmap, am observat o situatie care necesita o decizie imediata: variabilele experience_years si total_days_worked prezinta o corelatie pozitiva perfecta (1.00). Acest lucru era de asteptat, deoarece, in esenta, numarul de zile lucrate reprezinta o masuratoare directa a experientei. Pastrarea ambelor coloane in faza de modelare ar fi introdus redundanta, ingreunand inutil modelele. Prin urmare, decizia clara pentru preprocesare este eliminarea uneia dintre ele. Celelalte atribute numerice (skills_count, certifications, aggregated_score) nu arata corelatii semnificative intre ele, confirmand ca fiecare aduce informatie unica.

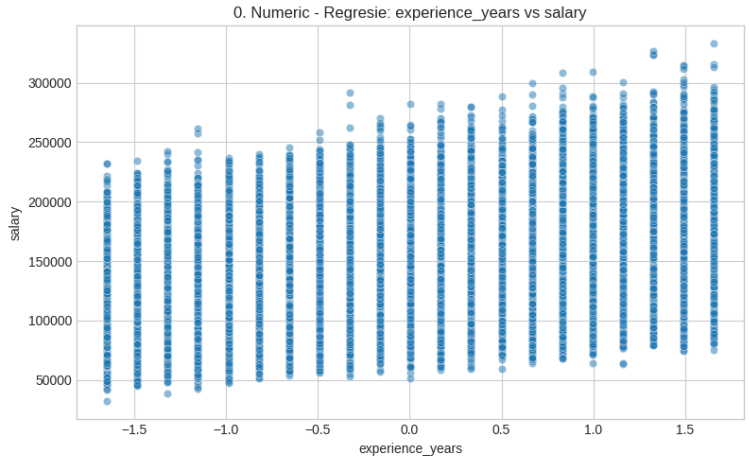
O a doua concluzie importanta extrasa din matricea Pearson se refera la relatia cu variabila tinta pentru regresie, salary. Am identificat o corelatie moderata, pozitiva (0.44), exclusiv cu variabila experience_years. Acest lucru se aliniaza cu asteptarile din piata muncii: salariul tinde sa creasca odata cu acumularea de experienta. In mod surprinzator, celelalte atribute numerice (numarul de skill-uri sau certificarile) au o corelatie extrem de slaba cu salariul (aproape de 0).

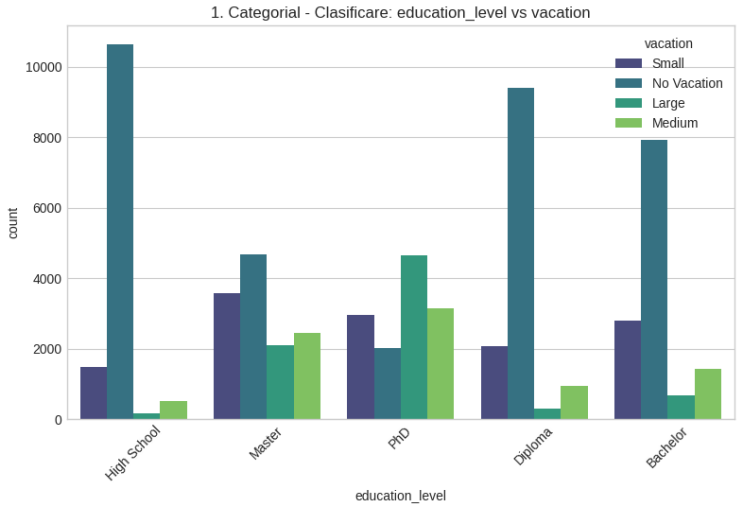
Corelatia categorica (Matricea Chi-Patrat): Coeficientul Pearson este limitat la variabile numerice. Prin urmare, pentru a testa redundanta intre atributele de tip text, am implementat un script care sa calculeze testul statistic Chi-Patrat pentru fiecare pereche posibila. Rezultatele (p-values) au fost vizualizate in a doua matrice, fiind setat un prag critic de 0.05 pentru a indica dependenta (marcat cu roz intens.

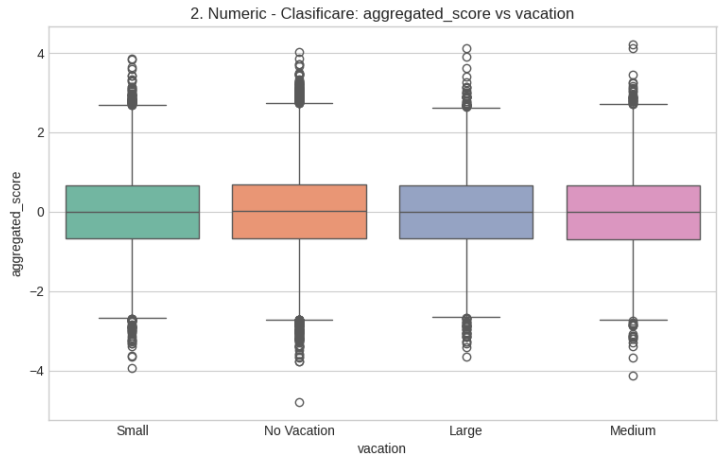
```
Verificarea redundantei intre atribute categorice (Chi-Patrat):
[job_title vs education_level] p-value: 0.9998 -> Independente
[job_title vs industry] p-value: 0.0767 -> Independente
[job_title vs company_size] p-value: 0.1566 -> Independente
[job_title vs location] p-value: 0.4865 -> Independente
[job_title vs remote_work] p-value: 0.2412 -> Independente
[job_title vs skill_bracket] p-value: 0.4177 -> Independente
[education_level vs industry] p-value: 0.9569 -> Independente
[education_level vs company_size] p-value: 0.1437 -> Independente
[education_level vs location] p-value: 0.6047 -> Independente
[education_level vs remote_work] p-value: 0.3550 -> Independente
[education_level vs skill_bracket] p-value: 0.1039 -> Independente
[industry vs company_size] p-value: 0.6637 -> Independente
[industry vs location] p-value: 0.1182 -> Independente
[industry vs remote_work] p-value: 0.0901 -> Independente
[industry vs skill_bracket] p-value: 0.4258 -> Independente
[company_size vs location] p-value: 0.2671 -> Independente
[company_size vs remote_work] p-value: 0.3125 -> Independente
[company_size vs skill_bracket] p-value: 0.2378 -> Independente
[location vs remote_work] p-value: 0.9402 -> Independente
[location vs skill_bracket] p-value: 0.5163 -> Independente
[remote_work vs skill_bracket] p-value: 0.8044 -> Independente
```

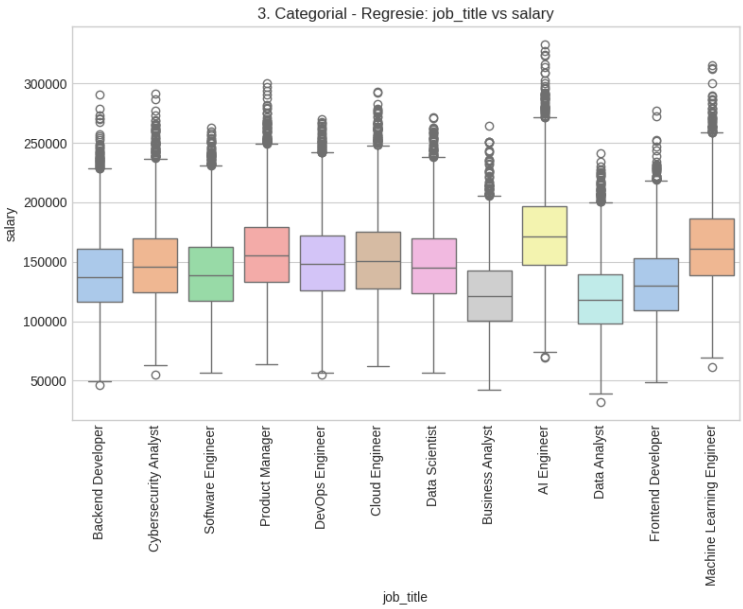
Teoria spune ca un p-value > 0.05 sugereaza respingerea ipotezei ca variabilele ar fi dependente. Asa cum evidentiaza graficul si afisarea din consola, cu exceptia diagonalei principale (unde variabilele se intersecteaza cu ele insele), toate combinatiile de predictori categorici (de exemplu, relatia dintre job_title si industry are un p-value = 0.0767) genereaza p-values peste pragul de 0.05. Concluzia este extrem de favorabila: atributele categorice sunt independente din punct de vedere statistic. Astfel, nu exista risc de redundanta pe aceasta zona, iar toate coloanele vor fi pastrate pentru faza de preprocesare si antrenare, fiecare aducand un plus de valoare modelului.

Analiza cerintelor minime









Am facut grafice pentru cerinta minima, luand pe rand toate cele 4 tipuri de analize pe care trebuia sa le fac.

Numeric - Regresie: experience_years vs salary – Am facut un Scatter Plot (grafic cu puncte) care cauta corelatia intre un atribut numeric si obiectivul de regresie. Pe axa OX sunt anii de experienta (standardizati intre -1.5 si 1.5), iar pe OY salariul in dolari.

Punctele urca usor spre dreapta, deci exista o corelatie pozitiva clara: cu cat creste experienta, cu atat creste si salariul. Deci ma astept la modelul de regresie sa am o linie aproape dreapta.

Categorial - Clasificare: education_level vs vacation- Am facut un Count Plot (histograma grupata) pentru a vedea distributia valorilor categoriale peste obiectivul de clasificare. Nivelul de educatie este pe orizontala si numarul de angajati pe verticala, colorati dupa tipul de vacanta (Small, Large, etc.).

Observ in el, de exemplu, ca cei cu "High School" au in mare parte "No Vacation" (bara albastra inalta), in timp ce la "PhD" apar mult mai multe vacante "Large" (bara verde). Asta ii spune modelului ca educatia e un predictor foarte bun pentru ce vacanta isi permite cineva.

Numeric - Clasificare: aggregated_score vs vacation - Acesta este un Boxplot care analizeaza un atribut numeric raportat la clasele de vacanta. Boxurile reprezinta plaja de valori unde stau 50% din date, linia din mijloc e mediana, iar punctele de sus si de jos sunt outliers.

Observ ca medienele scorurilor sunt cam la acelasi nivel (in jur de 0) pentru toate tipurile de vacanta. Asta sugereaza ca aggregated_score s-ar putea sa nu fie un predictor atat de puternic pentru vacanta precum este educatia, deoarece clasele nu sunt separate clar pe verticala.

Categorial - Regresie: job_title vs salary – Aici am facut tot un Boxplot, dar de data asta pentru a vedea cum influenteaza titlul jobului salariul. Aici se vede distributia salariilor pentru fiecare functie in parte (Data Scientist, AI Engineer, etc.).

Se vad diferentele mari de bani. De exemplu, "AI Engineer" are un box situat mult mai sus decat "Data Analyst", deci salariul mediu e mai mare. De asemenea, box-urile mai lungi indica faptul ca in acele domenii salariile variaza foarte mult, in timp ce cele scurte (ca la Data Analyst) au salarii mai standardizate.

In final, am creat un tabel de sinteza pentru a raspunde punctual cerintelor minime ale temei, testand impactul diferitelor tipuri de atribute asupra sarcinilor de predictie: salary (variabila tinta pentru regresie) si vacation (variabila tinta pentru clasificare).

	Relatie analizata	Metoda	Rezultat (p-value / Scor)	Concluzie
0	Numeric - Regresie (experience_years vs salary)	Pearson	0.44	Corelatie puternica; experienta este un factor determinant pentru salariu.
1	Categorial - Clasificare (education_level vs vacation)	Chi-Patrat (χ^2)	0.000	p < 0.05: Exista o dependenta statistica semnificativa intre educatie si vacanta.
2	Numeric - Clasificare (aggregated_score vs vacation)	Chi-Patrat (χ^2)*	0.423	p > 0.05: Variabilele sunt independente; scorul nu influenteaza tipul de vacanta.
3	Categorial - Regresie (job_title vs salary)	Chi-Patrat (χ^2)*	0.000	p < 0.05: Titlul jobului are o influenta majora asupra grilei de salarizare.

Pentru testele simple (numeric-numeric sau categoric-categoric), am folosit Pearson, respectiv Chi-Patrat. Totusi, pentru analizele mixte (numeric vs categoric), a trebuit sa gasesc o solutie ca sa pot aplica testul Chi-Patrat. Asa ca am discretizat variabilele numerice (le-am impartit in 4 intervale), transformandu-le temporar in categorii. Aceste teste sunt marcate cu o steluta (χ^2)* in tabel.

Numeric vs. Regresie (experience_years vs salary): Coeficientul Pearson este 0.44 (> 0.05). Deci este o corelatie pozitiva puternica, indicand clar ca experienta este un factor major in cresterea salariului, asa cum ma asteptam.

Categorial vs. Clasificare (education_level vs vacation): Am obtinut p-value = 0.000 (< 0.05). Deci exista o dependenta evidenta intre nivelul de educatie si tipul de vacanta pe care si-l permite un angajat.

Numeric vs. Clasificare (aggregated_score vs vacation): Aici p-value este 0.423 (> 0.05). Deci variabilele sunt complet independente.

Categorial vs. Regresie (job_title vs salary): p-value = 0.000 (< 0.05). Deci aici este prezenta o dependenta maxima. Functia detinuta influenteaza major grila de salarizare, fiind esential pentru viitorul model de regresie.

3.2 Preprocesarea datelor

1. **Imputarea valorilor lipsa**

In urma etapei de explorare a datelor (EDA), am identificat ca atributul categoric remote_work prezinta un deficit major de informatii, insumand 19.153 de inregistrari cu valoarea NaN (aproximativ 30% din intregul set de date).

Avand in vedere numarul foarte mare de inregistrari incomplete, reprezentand aproape o treime din setul de date, eliminarea acestor randuri ar fi dus la o pierdere foarte mare de informatii utile. Astfel, pentru tratarea acestor date lipsa, am utilizat clasa SimpleImputer din biblioteca scikit-learn.

Deoarece vorbim despre o variabila categorica, optiunile matematice clasice (media, mediana) nu pot fi aplicate. Am ales o imputare folosind strategia most_frequent (inlocuirea cu valoarea dominanta). Desi aceasta metoda introduce un oarecare dezechilibru artificial in distributia initiala a claselor "Yes/No/Hybrid", este o abordare standard solida pentru a obtine un set de date complet si functional pentru modelele de clasificare si regresie liniara, fara a crea noi categorii fictive.

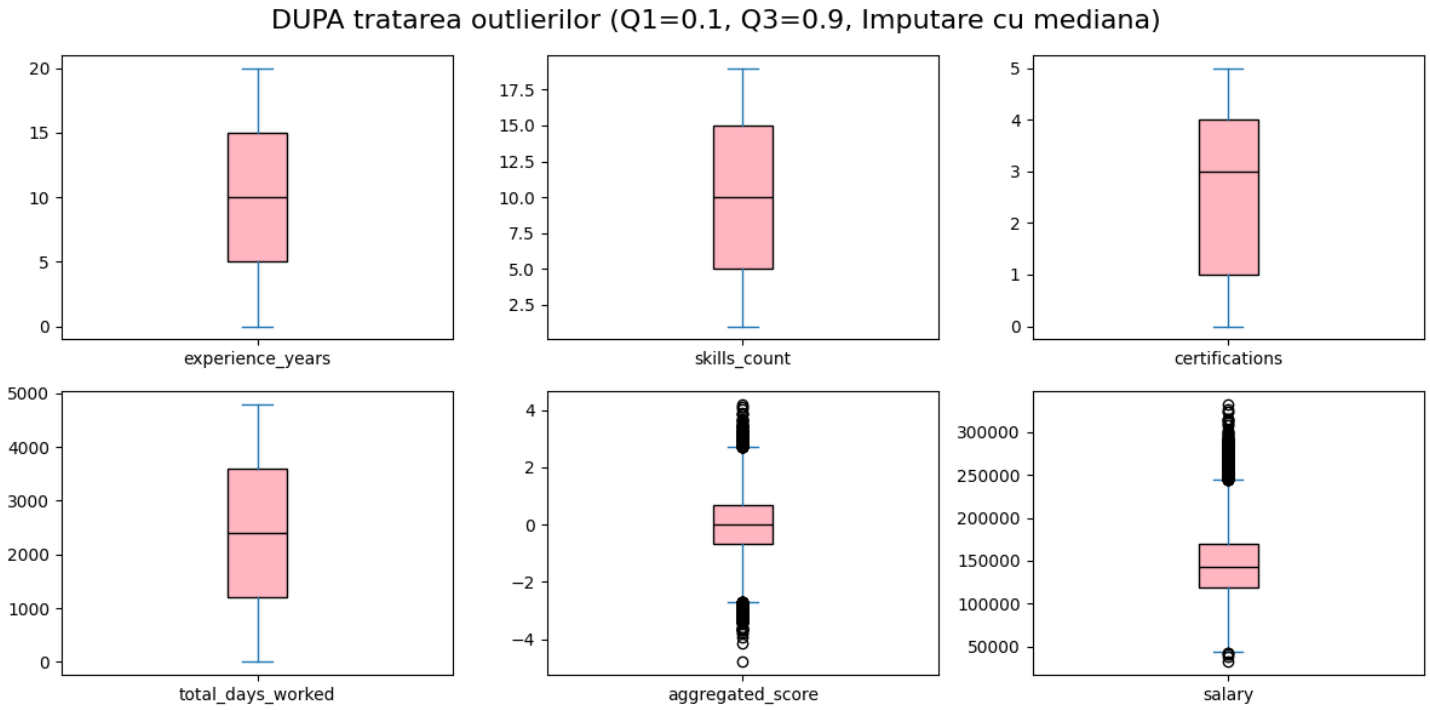
2. Determinarea si tratarea valorilor extreme

Pentru determinarea valorilor extreme, am analizat initial atributele numerice folosind metoda pandas.describe(). Analiza vizuala din etapa de explorare a indicat prezenta valorilor extreme in majoritatea atributelor numerice ale setului de date, in special in cazul atributului salary. Deoarece algoritmii de machine learning pot fi puternic distorsionati de aceste anomalii, am decis curatarea completa a tuturor coloanelor numerice.

Pentru a identifica valorile extreme, am utilizat metoda Inter-Quartile Range. In mod obisnuit, aceasta metoda foloseste cuartilele de 25% si 75%. Totusi, avand in vedere indicatiile din cerinta si faptul ca variabilele (precum salary) au o plaja foarte larga de valori valide, am ajustat intervalele quantilelor la valori mai extreme: Q1 = 0.1 si Q3 = 0.9. Aceasta ajustare mi-a permis sa disting corect intre adevaratele valori outlier si simplele valori mari, conservand astfel mai bine distributia naturala a datelor.

Valorile aflate in afara intervalului [Q1 - 1.5 * IQR, Q3 + 1.5 * IQR] au fost inregistrate ca anomalii si transformate in valori lipsa. Ulterior, pentru a imputa aceste date fara a fi afectat de alte variatii, am utilizat clasa SimpleImputer cu strategia "median".

Dupa rularea codului de curatare, am generat un nou set de grafice Boxplot pentru a verifica rezultatele. Este important de mentionat ca, desi anomaliile majore au fost eliminate cu succes prin regula noastra (0.1 - 0.9), functia standard de vizualizare Boxplot a bibliotecii pandas continua sa deseneze puncte negre folosind raportarea ei implicita (0.25 - 0.75). Prin urmare, punctele vizibile pe graficele finale de la salary sau aggregated_score nu mai sunt outliers, ci doar valori valide situate inspre extremele distributiei.



3. Eliminarea atributelor redundante

Analiza matricei de corelatie (Pearson) realizata la pasul 3.1 a scos in evidenta o corelatie perfecta, cu o valoare de 1.00, intre atributele numerice experience_years si total_days_worked.

Acest lucru indica o redundanta totala a datelor: cele doua coloane ofera exact aceeasi informatie modelului, adaugand complexitate computationala inutila fara a aduce o putere de predictie suplimentara.

Respectand indicatiile teoretice privind corelatiile, am decis sa pastrez atributul experience_years (deoarece s-a dovedit a fi un predictor esential si puternic corelat cu obiectivul de regresie, salariul) si sa elimin complet din setul de date atributul redundant today_days_worked. Astfel, algoritmul va beneficia de date mai curate si mai eficiente.

Pe de alta parte, in ceea ce priveste atributele categorice, testul Chi-Patrat aplicat in etapa de explorare a demonstrat ca acestea sunt complet independente intre ele (p-values mult peste 0.05). Prin urmare, nu a fost necesara eliminarea niciunei coloane de tip text, fiecare aducand o informatie unica modelului.

```
Eliminare atributel redundante
Coloane numerice inainte: ['experience_years', 'skills_count', 'certifications', 'total_days_worked', 'aggregated_score', 'salary']
Coloane numerice ramase: ['experience_years', 'skills_count', 'certifications', 'aggregated_score', 'salary']
```

4. Standardizarea datelor

Analizand plaja de valori a atributelor numerice ramase in setul de date, am observat diferente majore de scala. De exemplu, atributul salary contine valori de ordinul zecilor sau sutelor de mii, in timp ce atribute precum experience_years sau skills_count contin valori de ordinul zecilor sau unitatilor. In lipsa unei alinieri,

algoritmii sensibili la magnitudine (cum ar fi Regresia Logistica) ar acorda o importanta disproportionata atributelor cu valori absolut mai mari.

Pentru a preveni aceasta problema, am aplicat operatia de standardizare a atributelor numerice. Din biblioteca scikit-learn, am ales utilizarea clasei StandardScaler. Deoarece am tratat deja valorile extreme la pasul anterior, StandardScaler este o alegere optima. Acesta transforma datele din fiecare coloana astfel incat distributia rezultata sa aiba media 0 si deviatia standard 1, aducand toate atributele numerice la aceeasi scara de magnitudine.

Datele inainte de standardizare

	experience_years	skills_count	certifications	aggregated_score
0	19.0	12.0	0.0	-0.723308
1	4.0	13.0	5.0	-0.481551
2	8.0	16.0	4.0	-1.533057

Datele dupa standardizare

	experience_years	skills_count	certifications	aggregated_score
0	1.491845	0.374969	-1.468677	-0.724337
1	-0.987234	0.560459	1.455646	-0.482547
2	-0.326146	1.116930	0.870781	-1.534198

In urma standardizarii, datele brute si-au pierdut unitatile de masura originale (ani, dolari) si au fost transformate in scoruri Z. Aceste noi numere ii transmit modelului de Machine Learning distanta fiecarei inregistrari fata de media coloanei respective.

Asa cum se observa din rezultatele afisate:

- Pentru primul angajat (index 0), experienta sa reala de 19 ani a fost transformata in valoarea 1.49. Acest lucru ii spune algoritmului ca acest angajat are o experienta cu ~1.5 deviatii standard peste media generala a firmei. La fel, salariul sau de 203.997 a devenit 1.55 (mult peste salariul mediu).
- Pentru al doilea angajat (index 1), experienta sa de 4 ani a fost transformata in -0.98. Semnul minus ii indica modelului ca acest angajat se afla sub media firmei ca nivel de experienta.

Astfel, prin aceasta transformare, algoritmii vor judeca de acum atributele strict prin prisma performantei relative (cat de departe sunt de medie), eliminand complet bias-ul dat de marimea numerelor.

3.3 Utilizarea algoritmilor de Invatare Automata - Clasificare

1. Pregatirea datelor (Encodarea)

Inainte de a antrena orice model, a trebuit sa aduc datele la un format pe care algoritmii din scikit-learn sa il poata procesa (format strict numeric). In primul rand, am eliminat atributul salary din setul de date pentru

clasificare, deoarece acesta reprezinta variabila tinta pentru regresie si nu trebuie sa influenteze predictia tipului de vacanta. Pentru encodarea variabilelor, am folosit doua abordari diferite:

- Pentru variabila tinta (vacation): am folosit clasa LabelEncoder, care a transformat etichetele text (Small, Medium, Large, No Vacation) in valori numerice (0, 1, 2, 3). Am ales aceasta encodare pentru ca algoritmii de clasificare din biblioteca scikit-learn (inclusiv Decision Tree si Logistic Regression) asteapta ca variabila tinta sa fie un vector unidimensional (o singura coloana) care contine numere intregi, reprezentand clasa din care face parte exemplul.
- Pentru restul atributelor categorice predictor: am folosit abordarea de tip One-Hot Encoding (pd.get_dummies), utilizand parametrul drop_first=True pentru a evita redundanta (dummy variable trap). Am ales asta pt ca majoritatea atributelor categorice din setul de dat nu au o ordine naturala logica. Daca as fi folosit LabelEncoder pentru a le transforma in cifre (0, 1, 2, 3), un algoritm liniar precum Regresia Logistica ar fi interpretat gresit aceste valori. One-Hot Encoding rezolva aceasta problema transformand fiecare categorie intr-o coloana binara (0 sau 1), tratandu-le in mod absolut egal, fara a impune o relatie intre ele.

Atributele numerice au fost aduse la o scara comuna folosind StandardScaler. Acest pas a fost crucial, in special pentru functionarea optima a Regresiei Logistice.

La final, am impartit datele intr-un set de antrenare (80%) si unul de validare (20%), dupa cum am invatat la curs. Setul a capatat astfel o dimensiune de 45 de coloane, datorita expandarii prin One-Hot Encoding.

```
Pregatire date pt clasificare
Dimensiune X_train: (51200, 45)
Dimensiune X_val: (12800, 45)
```

2. Metodologia Cercetarii: Ablatia pe Baseline (Decision Tree)

Conform cerintelor, am stabilit ca baseline algoritmul Decision Tree Classifier. Pentru a intelege influenta hiperparametrilor si a imbunatati modelul, am realizat o ablatie urmarind doi parametri cheie: min_samples_leaf (numarul minim de exemple necesare pentru a forma o frunza) si max_depth (adancimea maxima a arborelui).

Clasificare DT - Ablatie pe 2 Hiperparametri

	Min_samples_leaf	Max_depth	Acuratete (↑)	Precizie (↑)	Recall (↑)	F1 (↑)
0	1	Nelimitat	0.673	0.676	0.673	0.675
1	5	Nelimitat	0.706	0.699	0.706	0.702
2	10	Nelimitat	0.727	0.716	0.727	0.721
3	20	Nelimitat	0.738	0.727	0.738	0.731
4	50	Nelimitat	0.751	0.742	0.751	0.746
5	**100**	**Nelimitat**	**0.751**	**0.743**	**0.751**	**0.747**
6	150	Nelimitat	0.748	0.743	0.748	0.745
7	1	5	0.669	0.682	0.669	0.652
8	1	10	0.742	0.735	0.742	0.738
9	1	20	0.679	0.681	0.679	0.680
10	1	50	0.673	0.676	0.673	0.675

In urma ablatiei am descoperit ca un arbore neconstrans (min_samples_leaf=1, max_depth=None) face overfit, obtinand o performanta de baza (F1-score ~0.67). Crescand valoarea lui min_samples_leaf, am constrans modelul sa generalizeze mai bine, iar performanta a atins un varf la min_samples_leaf=100 (F1 ~0.747).

Ablatia combinata: Pentru a face un model mai bun, am fixat pe rand un parametru la valoarea sa optima si l-am variat pe celalalt. Rezultatul acestei ablatii combinate a aratat ca cel mai bun model Decision Tree este obtinut cu configuratia min_samples_leaf=100 si max_depth=15 (F1-score = 0.747).

Clasificare DT - Ablatie Combinata (Min_samples_leaf = 100 si Max_depth = 10)

	Min_samples_leaf	Max_depth	Acuratete (↑)	Precizie (↑)	Recall (↑)	F1 (↑)
0	**100**	**15**	**0.751**	**0.743**	**0.751**	**0.747**
1	**100**	**50**	**0.751**	**0.743**	**0.751**	**0.747**
2	**100**	**20**	**0.751**	**0.743**	**0.751**	**0.747**
3	**100**	**Nelimitat**	**0.751**	**0.743**	**0.751**	**0.747**
4	50	10	0.750	0.741	0.750	0.745
5	150	10	0.744	0.740	0.744	0.742
6	100	10	0.746	0.738	0.746	0.742
7	20	10	0.745	0.736	0.745	0.740
8	5	10	0.744	0.737	0.744	0.740
9	1	10	0.742	0.735	0.742	0.738
10	100	5	0.668	0.683	0.668	0.652

3. Testarea algoritmilor avansati

Pentru a incerca sa depasesc performanta oferita de Decision Tree, am experimentat cu alti doi algoritmi discutati la curs: Random Forest Classifier (variind numarul de arbori) si Logistic Regression.

Testare si comparare alte modele

	Model	Configuratie	Acuratete (↑)	Precizie (↑)	Recall (↑)	F1 (↑)
0	Decision Tree (Leaf=100)	Baseline optimizat	0.751	0.743	0.751	0.747
1	Random Forest	n_estimators=10	0.710	0.686	0.710	0.694
2	Random Forest	n_estimators=50	0.739	0.719	0.739	0.726
3	Random Forest	n_estimators=100	0.742	0.723	0.742	0.730
4	Random Forest	n_estimators=150	0.744	0.726	0.744	0.732
5	**Logistic Regression**	**Standard (Liniar)**	**0.764**	**0.751**	**0.764**	**0.755**

In urma tabelului am decoperit:

Random Forest: Eliminand constrangerile prea stricte pe frunze si lasand arborii sa creasca natural, modelul a atins un scor F1 de ~0.732 (pentru n_estimators=150). Desi este o imbunatatire semnificativa, nu a reusit sa depaseasca performanta obtinuta de modelele concurente pe acest set de date.

Logistic Regression: Setata cu max_iter=4000 pentru a asigura convergenta si antrenata pe datele standardizate (inainte aveam mai putine iteratii si aveam o eroare, am marit nr de iteratii pana am rezolvat eroarea), a reusit sa depaseasca Decision Tree-ul. Raportandu-ne strict la metricile robuste la dezechilibru de clase, Regresia Logistica a obtinut cel mai bun scor F1 Weighted de 0.755, oferind cel mai bun echilibru intre Precizie si Recall.

4. Evaluarea Algoritmilor si Analiza Calitativa

Pentru a intelege exact ce gresesc modelele si de ce scorul F1 se opreste in jurul valorii de 0.75, am realizat un tabel comparativ detaliat per clasa si am generat matricele de confuzie pentru cele mai bune doua modele identificate: Decision Tree optimizat (Leaf=100, Depth=15) si Logistic Regression.

Analiza per Clasa: Decision Tree (Leaf=100, Depth=15)

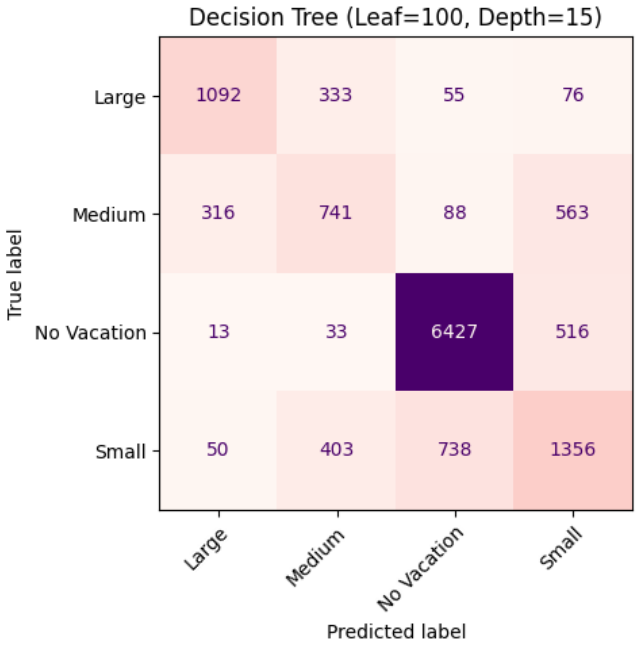
	precision	recall	f1-score	support
Large	0.74	0.70	0.72	1556
Medium	0.49	0.43	0.46	1708
No Vacation	0.88	0.92	0.90	6989
Small	0.54	0.53	0.54	2547
accuracy			0.75	12800
macro avg	0.66	0.65	0.65	12800
weighted avg	0.74	0.75	0.75	12800

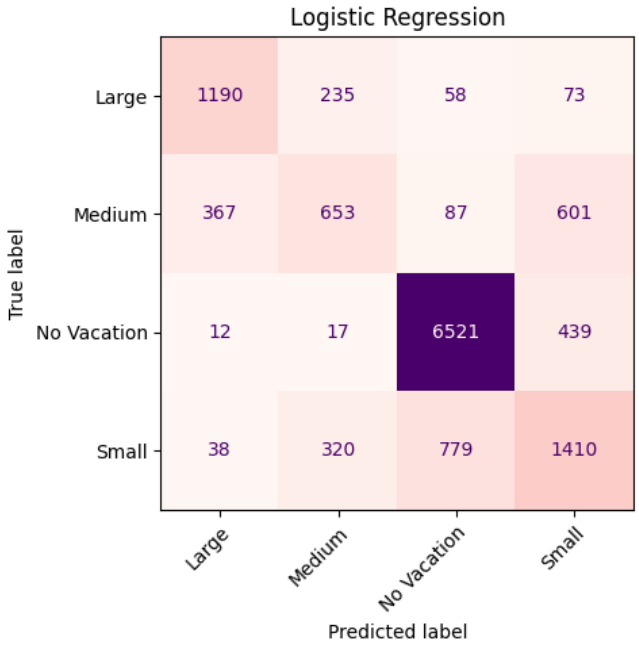
Analiza per Clasa: Logistic Regression

	precision	recall	f1-score	support
Large	0.74	0.76	0.75	1556
Medium	0.53	0.38	0.45	1708
No Vacation	0.88	0.93	0.90	6989
Small	0.56	0.55	0.56	2547
accuracy			0.76	12800
macro avg	0.68	0.66	0.66	12800
weighted avg	0.75	0.76	0.75	12800

Generand raportul de clasificare, am observat urmatoarele comportamente:

- Impactul dezechilibrului de date (Coloana Support): Coloana support arata numarul real de inregistrari din setul de validare. Se observa un dezechilibru masiv: clasa "No Vacation" domina cu aproape 7000 de exemple (din 12800), in timp ce restul claselor au intre 1500 si 2550.
- Performanta excelenta pe extreme: Ambele modele exceleaza in a prezice clasa majoritara "No Vacation", obtinand un scor F1 impresionant de 0.90. De asemenea, algoritmii recunosc destul de bine si vacantele scumpe ("Large"), Regresia Logistica obtinand aici un F1 de 0.75.
- Dificultatea claselor de mijloc: Raportul confirma matematic vulnerabilitatea modelelor la clasele intermediare. Pentru clasa "Medium", F1-score-ul se prabuseste la 0.44 - 0.46 (cu un recall de doar 0.38 - 0.43, ceea ce inseamna ca rateaza mai mult de jumatate din vacantele Medium reale). Clasa "Small" ramane blocata la un F1 de ~0.54 - 0.56. Aceasta este dovada clara a faptului ca atributele noastre nu sunt suficient de descriptive pentru zona de tranzit intre tipurile de vacanta.
- Capcana Acuratetei: Desi acuratetea generala (Accuracy) este de ~0.76, media neponderata (Macro Avg) cade la 0.66. Acest lucru demonstreaza ca scorul general mare este sustinut artificial de volumul urias al predictiilor corecte din clasa "No Vacation".





Matricele de confuzie confirma vizual aceasta dificultate intre clasele de mijloc. Privind modelul Logistic Regression:

- Confuzia Medium - Small: Din persoanele care au avut in realitate o vacanta "Medium", modelul a prezis corect doar 653 si a clasificat gresit 601 persoane ca avand o vacanta "Small".
- Confuzia Small - No Vacation: Din persoanele cu vacanta "Small", modelul a etichetat gresit un numar urias de 779 de angajati ca "No Vacation".
- Clasa "Large" se confunda cel mai des cu "Medium" (235 cazuri gresite).

	Model	Acuratete	Precizie (Large)	Recall (Large)	F1 (Large)	Precizie (Medium)	Recall (Medium)	F1 (Medium)	Precizie (No Vacation)	Recall (No Vacation)	F1 (No Vacation)	Precizie (Small)	Recall (Small)	F1 (Small)
0	Decision Tree (Leaf=100, Depth=15)	0.751	**0.742**	0.702	0.722	0.491	**0.434**	**0.461**	**0.879**	0.920	0.899	0.540	0.532	0.536
1	Logistic Regression	**0.764**	0.741	**0.765**	**0.752**	**0.533**	0.382	0.445	0.876	**0.933**	**0.904**	**0.559**	**0.554**	**0.556**

Analizand tabelul comparativ, construit respectand cerinta de a releva prin bolduire valorile maxime pentru fiecare metrica, am extras urmatoarele concluzii valoroase despre diferentele subtile de comportament dintre cei doi algoritmi:

- Acuratetea generala si Dominatia Regresiei Logistice: Regresia Logistica castiga detasat comparatia directa, obtinand o acuratete superioara (0.764 fata de 0.751). Daca urmarim metrica F1, care este cea mai relevanta pentru seturi dezechilibrate, Regresia Logistica este castigatoarea pe 3 din cele 4 clase: "Large" (0.752 vs 0.722), "No Vacation" (0.904 vs 0.899) si "Small" (0.556 vs 0.536). Acest lucru demonstreaza ca abordarea probabila si liniara a regresiei a reusit sa separe mai bine pachetele de vacanta decat regulile stricte de ramificare ale arborelui.
- Performanta pe clasa "Large": Desi Arborele de Decizie este extrem de putin mai precis (0.742 vs 0.741), Regresia Logistica este capabila sa gaseasca mult mai multe vacante "Large" din totalul celor

reale, avand un Recall net superior (0.765 vs 0.702), ceea ce o face mult mai utila in practica pentru aceasta categorie.

- Cazul special al clasei "Medium" (Singura victorie a Arborelui): Clasa "Medium" este singura zona unde Arborele de Decizie obtine un scor F1 mai mare (0.461 fata de 0.445). Motivul este dat de Recall: Arborele reuseste sa identifice 43.4% din vacantele Medium reale, in timp ce Regresia Logistica rateaza masiv aici (doar 38.2% Recall). Totusi, ambele scoruri raman foarte slabe, confirmand din nou lipsa de separabilitate clara a acestei clase fata de celelalte in spatiul atributelor.
- Clasa majoritara "No Vacation": Aici lupta este foarte stransa. Arborele este usor mai precis, dar Regresia Logistica greseste mai rar (Recall de 0.933), avand scorul F1 de peste 0.9.

Concluzie finala pentru clasificare

Bolduirea metricilor in tabel evidentiaza vizual superioritatea Regresiei Logistice. Prin aplicarea corecta a One-Hot Encoding si a standardizarii pe atributele numerice, modelul a reusit sa fie mai eficient decat Arborele de Decizie, fiind alegerea finala recomandata pentru acest set de date.

3.4 Utilizarea algoritmilor de Invatare Automata - Regresie

1. Pregatirea datelor si Preprocesarea

Pentru partea de regresie, scopul meu a fost sa prezic atributul numeric salary. Primul lucru pe care l-am facut a fost sa elimin coloana vacation (tinta de la clasificare), pentru ca nu are ce cauta in input-ul modelului de regresie.

Dupa ce am facut asta, am aplicat One-Hot Encoding pentru toate variabilele categorice ramase (job_title, education_level, etc.) ca sa le transform in cifre pe care modelul sa le poata procesa. Un pas foarte important cerut in enunt a fost standardizarea. Acest pas s-a facut la etapa de preprocesare, doar ca eu acolo am folosit functia de standardizare si pe salariu, iar in acest par mi-am dat seama ca nu ar trebui, lasand salariul asa cum era initial. Deci am folosit StandardScaler doar pe atributele numerice de intrare (X), nu si pe salariu (y). Pentru ca daca standardizam si salariul, erorile (RMSE) dadeau valori de genul 0.20, care nu ajutau sa compar cu pragurile din enunt (unde se vorbeste de valori de ordinul miilor). Asa, pastrand salariul in formatul lui original, am obtinut rezultate realiste.

2. Metodologia Cercetarii: Baseline si Regularizare

Conform cerintei minime, am stabilit un baseline folosind modelul clasic de LinearRegression din sklearn.

Ulterior, pentru a experimenta cu factorul de regularizare, am decis sa folosesc Ridge Regression (Regularizare L2). Am ales acest model pentru ca stiam din curs ca ajuta atunci cand modelul devine prea complex. Am realizat un set de experimente (o ablatie) variind hiperparametrul alpha (forta de regularizare) intre valorile 0.1, 1, 10, 100 si 1000, ca sa vad cum se modifica eroarea.

3. Evaluarea Algoritmilor si Analiza Rezultatelor

```
Linear Regression (No Regularization)
Train RMSE: 7343.87
Validation RMSE: 7390.99

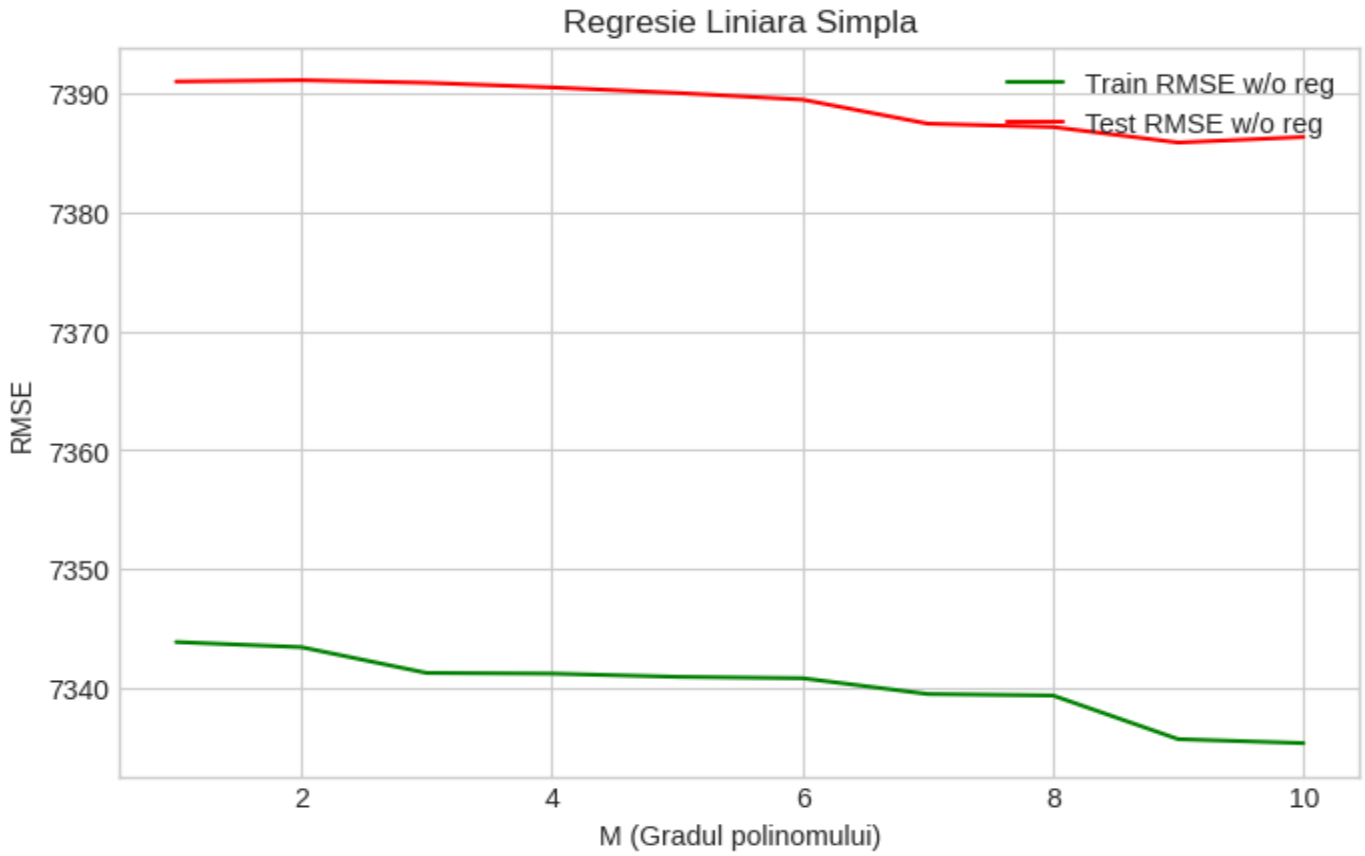
Ridge regression (L2 Regularization)
Alpha: 0.1    | Train RMSE: 7343.87 | Val RMSE: 7391.01
Alpha: 1      | Train RMSE: 7343.91 | Val RMSE: 7391.27
Alpha: 10     | Train RMSE: 7347.99 | Val RMSE: 7397.40
Alpha: 100    | Train RMSE: 7618.60 | Val RMSE: 7681.86
Alpha: 1000   | Train RMSE: 11405.52 | Val RMSE: 11477.44
```

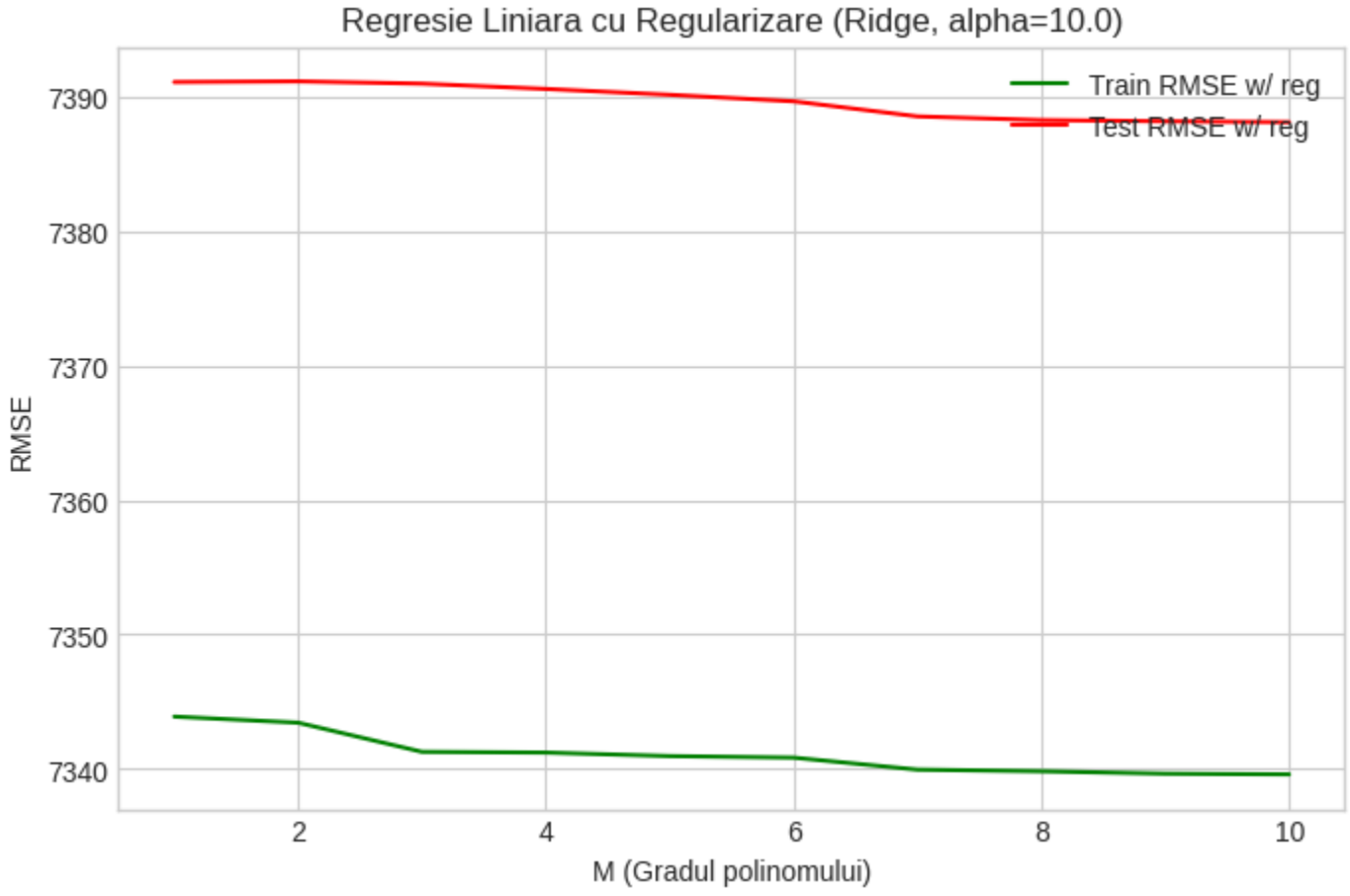
```
Legenda: (↓) O valoare mai mica este mai buna.
```

Model	Regularization (α)	Train RMSE (↓)	Validation RMSE (↓)	MAE	MSE	R2
Linear Regression (Baseline)	None	**7343.87**	**7390.99**	**5637.43**	**54626697.04**	**0.96**
Ridge Regression	0.1	**7343.87**	7391.01	5637.46	54627056.47	**0.96**
	1	7343.91	7391.27	5637.75	54630851.55	**0.96**
	10	7347.99	7397.40	5643.40	54721557.73	**0.96**
	100	7618.60	7681.86	5856.40	59010914.77	**0.96**
	1000	11405.52	11477.44	8456.04	131731530.85	0.91

Dupa ce am rulat toate modelele, am centralizat rezultatele in tabelul de mai sus. Se observa clar ca Regresia Liniara si Ridge (cu alpha mic) sunt castigatoarele, avand cel mai mic RMSE (~7343) si cel mai mare R2 (0.96).

Analiza Grafica : Variatia gradului polinomului (M) si Complexitatea





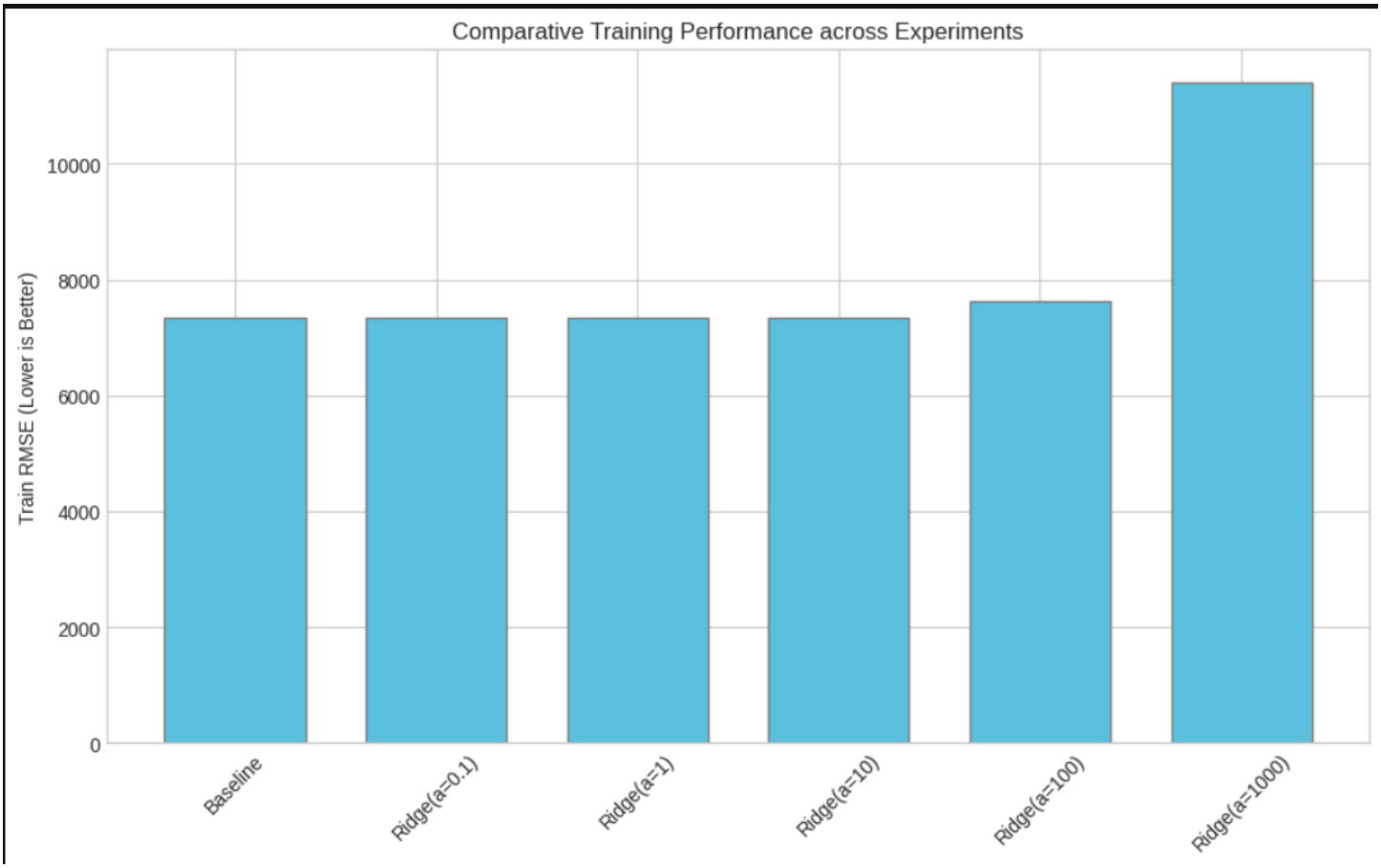
In aceste doua grafice, am explorat complexitatea modelului prin variatia gradului polinomului (M), de la 1 la 10, urmand metodologia de studiu din laborator.

Scopul a fost sa vad daca transformarea datelor prin adaugarea de trasaturi polinomiale (ridicarea la putere a atributelor) ajuta modelul sa gaseasca relatii mai complexe intre trasaturi si salariu. In teorie, un grad M mai mare ar trebui sa permita modelului sa nu aiba chiar o linie dreapta, fiind una mai stramba, dar risca sa duca la overfitting masiv.

Spre deosebire de exemplele teoretice din laborator, unde la un M mare eroarea pe test explodeaza, aici am observat ca liniile de RMSE raman aproape plate atat pentru Regresia Simpla, cat si pentru Ridge.

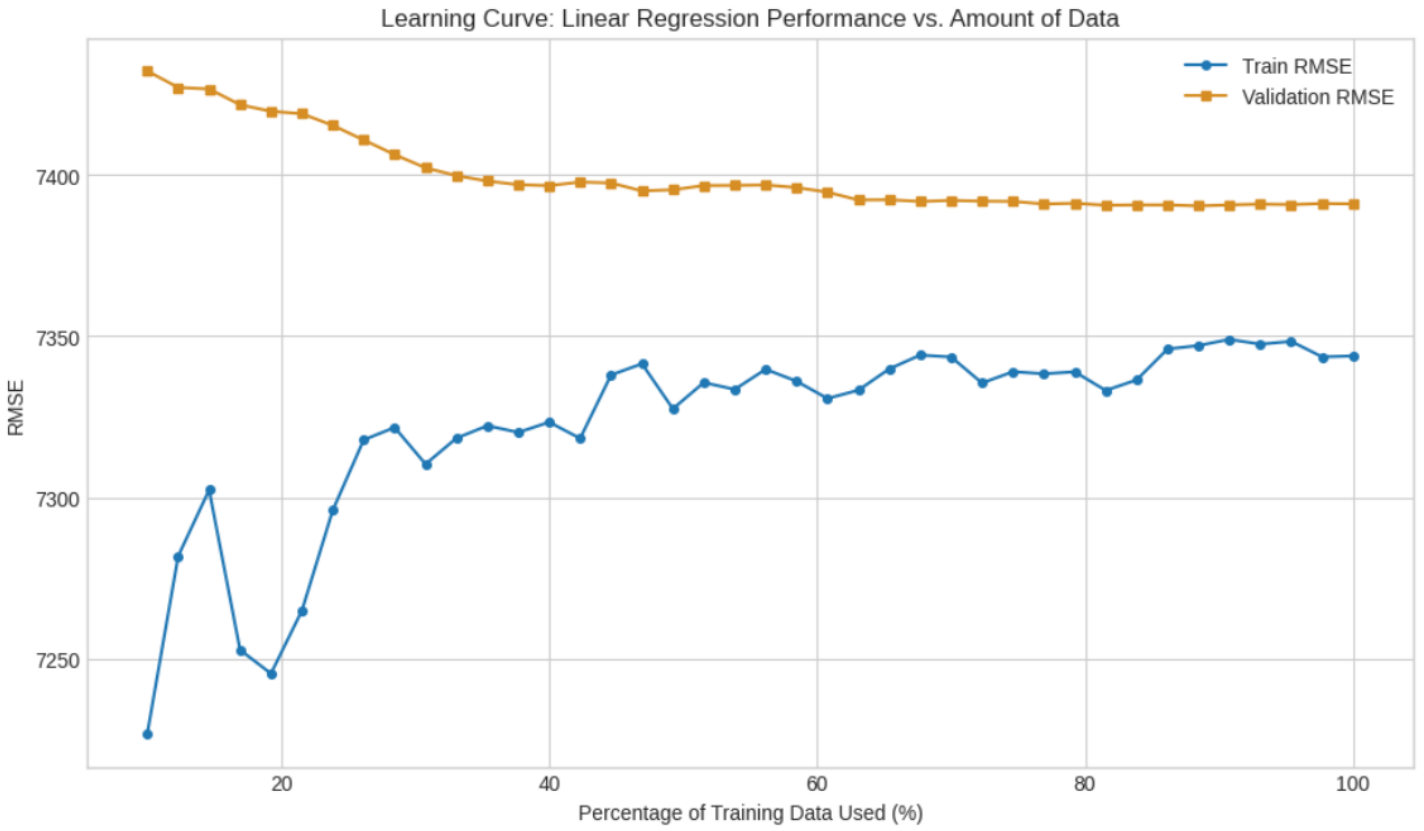
Acest comportament imi confirma inca o data ca relatia din datele mele este una nativ liniara. Deoarece setul de date este mare si trasaturile au fost preprocesate (multe fiind de tip 0/1 din One-Hot Encoding), ridicarea lor la puteri superioare nu aduce informatie noua si nici nu pacaleste modelul sa faca overfit. Modelul ramane stabil si robust indiferent de complexitatea polinomiala aplicata, ceea ce valideaza alegerea regresiei liniare ca fiind solutia corecta si suficienta pentru aceasta problema.

Analiza Grafica 1: Performanta la antrenare



In aceasta figura am suprapus performantele de antrenament pentru a le compara usor, fara sa incarc graficul cu prea multe curbe ilizibile. Se vede ca eroarea de antrenament ramane aproape neschimbata pentru majoritatea configuratiilor, ceea ce inseamna ca modelul este foarte stabil pe aceste date. Cresterea usoara de la final (alpha=1000) arata ca o regularizare prea mare incepe sa strice modelul (underfit).

Analiza Grafica 2: Overfit si Curba de Invatare



Pentru a vedea daca modelul face overfit, am ales sa reprezint curba de invatare pentru cel mai bun model (Linear Regression). Am pus pe acelasi grafic curba de Train si cea de Validation in functie de cantitatea de date folosita.

Concluzia a fost ca graficul arata ideal. La inceput (sub 20% date), distanta dintre Train si Validare este mare, semn ca modelul nu are destule exemple. Dar, pe masura ce ajung la 100% din date, cele doua curbe converg si raman paralele, cu un spatiu foarte mic intre ele. Asta demonstreaza clar ca modelul nu face overfit si generalizeaza foarte bine pe date pe care nu le-a vazut inainte.

4. Concluzii finale asupra sarcinii de Regresie

In urma tuturor experimentelor rulate, pot afirma cu incredere ca Regresia Liniara (Baseline), alaturi de Ridge Regression (cu un alpha mic), reprezinta solutia optima pentru acest set de date.

Motivul principal tine de structura matematica a problemei. Salariile nu sunt de obicei distribuite haotic; ele urmeaza o logica de business clara, adesea bazata pe o grila salariala. In acest dataset, salariul pare sa fie o suma ponderata de factori: ani de experienta, nivelul de educatie si numarul de certificari. Regresia Liniara este exact instrumentul care invata aceste ponderi si le aduna pentru a scoate rezultatul final.

Analiza Regularizarii (Ridge): Desi am testat Ridge pentru a preveni un potential overfitting, am observat ca rezultatele au fost aproape identice cu cele ale regresiei simple pentru valori mici ale lui alpha (0.1, 1, 10). Acest lucru imi confirma faptul ca setul de date este curat si nu are zgomot excesiv care sa pacaleasca modelul, sau ca le-am eliminat eficient. Curba de invatare a aratat oricum o convergenta perfecta, deci nu a fost nevoie de o penalizare agresiva a coeficientilor pentru a forta generalizarea.